May 4, 2026

[1]:
```
# ============================================================
# RoBERTa Binary Classification with 5-Fold CV + Final Learning Curves
# Local RoBERTa Model Version: No Online Download
# Dataset columns:
# Rating_Star, Headings, Base_Reviews, Have_issue
# No bs4 / BeautifulSoup required
# ============================================================

import os
import re
import random
import warnings
from collections import Counter
from copy import deepcopy

import pandas as pd
import numpy as np

import torch
from torch import nn
from torch.utils.data import DataLoader, Dataset
from torch.optim import AdamW

from transformers import (
    RobertaTokenizerFast,
    RobertaModel,
    get_linear_schedule_with_warmup
)

from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    roc_curve,
    auc,
    precision_recall_fscore_support
```

```
)

from imblearn.over_sampling import RandomOverSampler

import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm

# ============================================================
# 0. Settings
# ============================================================

warnings.filterwarnings("ignore")

SEED = 42

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

# ============================================================
# 1. Local Paths
# ============================================================

LOCAL_ROBERTA_PATH = "/root/.ssh/Issue_Detection_Paper_data/
 ↪bert_best_final_model/roberta"

dataset_path = "/root/.ssh/Issue_Detection_Paper_data/
 ↪annotated_dataset_for_binary_classifications.csv"

OUTPUT_DIR = "/root/.ssh/Issue_Detection_Paper_data/roberta_binary_outputs"

SAVE_DIR = "/root/.ssh/Issue_Detection_Paper_data/roberta_binary_final_model"

os.makedirs(OUTPUT_DIR, exist_ok=True)
os.makedirs(SAVE_DIR, exist_ok=True)

if not os.path.isdir(LOCAL_ROBERTA_PATH):
    raise FileNotFoundError(
        f"Local RoBERTa model folder not found: {LOCAL_ROBERTA_PATH}"
    )

if not os.path.isfile(dataset_path):
    raise FileNotFoundError(
```

```
        f"Dataset CSV file not found: {dataset_path}"
    )

required_roberta_files = [
    "config.json",
    "vocab.json",
    "merges.txt"
]

for file_name in required_roberta_files:
    file_path = os.path.join(LOCAL_ROBERTA_PATH, file_name)
    if not os.path.isfile(file_path):
        raise FileNotFoundError(
            f"Missing required RoBERTa file: {file_path}"
        )

weight_file_bin = os.path.join(LOCAL_ROBERTA_PATH, "pytorch_model.bin")
weight_file_safe = os.path.join(LOCAL_ROBERTA_PATH, "model.safetensors")

if os.path.isfile(weight_file_bin):
    USE_SAFETENSORS = False
    print("  Found pytorch_model.bin. The code will use pytorch_model.bin.")
elif os.path.isfile(weight_file_safe):
    USE_SAFETENSORS = True
    print("  Found model.safetensors. The code will use model.safetensors.")
else:
    raise FileNotFoundError(
        "No RoBERTa weight file found. "
        "Please add either pytorch_model.bin or model.safetensors."
    )

print(f"  Using local RoBERTa model from: {LOCAL_ROBERTA_PATH}")
print(f"  Using dataset from: {dataset_path}")

# ============================================================
# 2. Device
# ============================================================

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(f"Using device: {device}")

# ============================================================
# 3. Load & Inspect Dataset
# ============================================================

try:
```

```
    df = pd.read_csv(dataset_path, encoding="utf-8")
except UnicodeDecodeError:
    df = pd.read_csv(dataset_path, encoding="latin1")

print(" Dataset loaded. Original columns:", df.columns.tolist())

# ------------------------------------------------------------
# Rename only if older column names exist.
# Your current dataset already has Base_Reviews and Have_issue.
# ------------------------------------------------------------

df = df.rename(
    columns={
        "comment_Text": "Base_Reviews",
        "Review": "Base_Reviews",
        "Reviews": "Base_Reviews",
        "Have issue": "Have_issue"
    }
)

required_columns = ["Base_Reviews", "Have_issue"]

for col in required_columns:
    if col not in df.columns:
        raise ValueError(
            f"Missing required column in dataset: {col}. "
            f"Available columns are: {df.columns.tolist()}"
        )

# ============================================================
# 4. Convert Labels
# ============================================================

def convert_issue(issue):
    issue = str(issue).lower().strip()

    if issue in ["no", "0", "no issue", "false", "none"]:
        return 0
    else:
        return 1

df["Have_issue"] = df["Have_issue"].apply(convert_issue)

df["Base_Reviews"] = df["Base_Reviews"].fillna("")

print("\nClass distribution original:")
print(df["Have_issue"].value_counts())
```

```
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="Have_issue")
plt.title("Original Class Distribution")
plt.xticks([0, 1], ["No issue", "Have_issue"])
plt.xlabel("Class")
plt.ylabel("Count")
plt.tight_layout()

class_dist_path = os.path.join(
    OUTPUT_DIR,
    "roberta_binary_original_class_distribution.png"
)

plt.savefig(class_dist_path, dpi=300, bbox_inches="tight")
plt.show()

print(f"  Class distribution figure saved to: {class_dist_path}")

# ============================================================
# 5. Preprocessing
# No bs4 / BeautifulSoup required
# ============================================================

def simple_fix_contractions(text):
    contraction_map = {
        "can't": "cannot",
        "won't": "will not",
        "n't": " not",
        "'re": " are",
        "'s": " is",
        "'d": " would",
        "'ll": " will",
        "'t": " not",
        "'ve": " have",
        "'m": " am"
    }

    text = str(text)

    for key, value in contraction_map.items():
        text = text.replace(key, value)
        text = text.replace(key.upper(), value.upper())

    return text

def preprocess_review(text):
```

```
    if not isinstance(text, str):
        text = str(text)

    text = simple_fix_contractions(text)

    # Remove HTML tags without BeautifulSoup
    text = re.sub(r"<.*?>", " ", text)

    # Remove URLs
    text = re.sub(r"http\S+|www\S+", " ", text)

    # Keep only letters, numbers, and spaces
    text = re.sub(r"[^a-zA-Z0-9\s]", "", text)

    # Remove extra spaces
    text = " ".join(text.split())

    return text

df["ReviewP"] = df["Base_Reviews"].apply(preprocess_review)

texts = df["ReviewP"].tolist()
labels = df["Have_issue"].tolist()

# ============================================================
# 6. Train/Test Split
# ============================================================

X_temp, X_test, y_temp, y_test = train_test_split(
    texts,
    labels,
    test_size=0.15,
    random_state=SEED,
    stratify=labels
)

X_train_raw = np.array(X_temp)
y_train_raw = np.array(y_temp)

print("\nDataset Splits:")
print(f"→ Full Train for CV: {len(X_train_raw)} samples")
print(f"→ Held-Out Test: {len(X_test)} samples")
print(f"→ Train class raw: {Counter(y_train_raw)}")
print(f"→ Test class: {Counter(y_test)}")

# ============================================================
# 7. Tokenizer
```

```python
# ============================================================

tokenizer = RobertaTokenizerFast.from_pretrained(
    LOCAL_ROBERTA_PATH,
    local_files_only=True
)

print(" Local RoBERTa tokenizer loaded successfully")

# ============================================================
# 8. Dataset Class
# ============================================================

class TextClassificationDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = str(self.texts[idx])
        label = int(self.labels[idx])

        encoding = self.tokenizer(
            text,
            truncation=True,
            padding="max_length",
            max_length=self.max_length,
            return_tensors="pt",
            return_attention_mask=True
        )

        return {
            "input_ids": encoding["input_ids"].flatten(),
            "attention_mask": encoding["attention_mask"].flatten(),
            "label": torch.tensor(label, dtype=torch.long)
        }

# ============================================================
# 9. RoBERTa Classifier
# ============================================================

class RoBERTaClassifier(nn.Module):
```

```
    def __init__(self, model_path, num_classes):
        super().__init__()

        self.roberta = RobertaModel.from_pretrained(
            model_path,
            local_files_only=True,
            use_safetensors=USE_SAFETENSORS
        )

        self.dropout = nn.Dropout(0.1)
        self.fc = nn.Linear(self.roberta.config.hidden_size, num_classes)

    def forward(self, input_ids, attention_mask):
        outputs = self.roberta(
            input_ids=input_ids,
            attention_mask=attention_mask
        )

        cls_output = outputs.last_hidden_state[:, 0, :]
        x = self.dropout(cls_output)
        logits = self.fc(x)

        return logits

# ============================================================
# 10. Hyperparameters
# ============================================================

model_name = LOCAL_ROBERTA_PATH

num_classes = 2
max_length = 128
batch_size = 16
num_epochs = 6
learning_rate = 1e-5

print("\nHyperparameters:")
print(f"Model path: {model_name}")
print(f"Number of classes: {num_classes}")
print(f"Max length: {max_length}")
print(f"Batch size: {batch_size}")
print(f"Epochs: {num_epochs}")
print(f"Learning rate: {learning_rate}")

# ============================================================
# 11. 5-Fold Cross Validation
# ============================================================
```

```
print("\nStarting 5-Fold CV...")

k_folds = 5

skf = StratifiedKFold(
    n_splits=k_folds,
    shuffle=True,
    random_state=SEED
)

cv_results = []

all_train_accuracies = []
all_val_accuracies = []

for fold, (train_idx, val_idx) in enumerate(
    skf.split(X_train_raw, y_train_raw),
    1
):
    print(f"\n{'=' * 50}")
    print(f"FOLD {fold}/{k_folds}")
    print(f"{'=' * 50}")

    X_tr = X_train_raw[train_idx]
    X_val = X_train_raw[val_idx]

    y_tr = y_train_raw[train_idx]
    y_val = y_train_raw[val_idx]

    ros = RandomOverSampler(random_state=SEED)

    X_tr_res, y_tr_res = ros.fit_resample(
        X_tr.reshape(-1, 1),
        y_tr
    )

    X_tr_res = X_tr_res.flatten().tolist()
    y_tr_res = y_tr_res.tolist()

    print(
        f"Fold {fold} → Train: {len(X_tr_res)} oversampled, "
        f"Val: {len(X_val)}"
    )
    print(f"→ Train classes: {Counter(y_tr_res)}")
    print(f"→ Val classes: {Counter(y_val)}")
```

```
    train_dataset = TextClassificationDataset(
        X_tr_res,
        y_tr_res,
        tokenizer,
        max_length
    )

    val_dataset = TextClassificationDataset(
        X_val.tolist(),
        y_val.tolist(),
        tokenizer,
        max_length
    )

    train_loader = DataLoader(
        train_dataset,
        batch_size=batch_size,
        shuffle=True
    )

    val_loader = DataLoader(
        val_dataset,
        batch_size=batch_size,
        shuffle=False
    )

    model = RoBERTaClassifier(
        model_name,
        num_classes
    ).to(device)

    optimizer = AdamW(
        model.parameters(),
        lr=learning_rate
    )

    total_steps = len(train_loader) * num_epochs

    scheduler = get_linear_schedule_with_warmup(
        optimizer,
        num_warmup_steps=0,
        num_training_steps=total_steps
    )

    criterion = nn.CrossEntropyLoss()

    fold_train_accs = []
```

```
    fold_val_accs = []

    best_val_acc = 0.0
    best_state = None

    best_val_true = None
    best_val_preds = None

    for epoch in range(num_epochs):
        model.train()

        total_loss = 0.0
        correct = 0
        total = 0

        for batch in train_loader:
            optimizer.zero_grad()

            input_ids = batch["input_ids"].to(device)
            attention_mask = batch["attention_mask"].to(device)
            labels_batch = batch["label"].to(device)

            outputs = model(
                input_ids,
                attention_mask
            )

            loss = criterion(
                outputs,
                labels_batch
            )

            loss.backward()

            torch.nn.utils.clip_grad_norm_(
                model.parameters(),
                1.0
            )

            optimizer.step()
            scheduler.step()

            total_loss += loss.item()

            preds = outputs.argmax(dim=1)

            correct += (preds == labels_batch).sum().item()
```

```
            total += labels_batch.size(0)

        train_acc = correct / max(1, total)

        fold_train_accs.append(train_acc)

        model.eval()

        val_preds = []
        val_true = []

        with torch.no_grad():
            for batch in val_loader:
                input_ids = batch["input_ids"].to(device)
                attention_mask = batch["attention_mask"].to(device)
                labels_batch = batch["label"].to(device)

                outputs = model(
                    input_ids,
                    attention_mask
                )

                preds = outputs.argmax(dim=1)

                val_preds.extend(preds.cpu().numpy())
                val_true.extend(labels_batch.cpu().numpy())

        val_acc = accuracy_score(
            val_true,
            val_preds
        )

        fold_val_accs.append(val_acc)

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            best_state = deepcopy(model.state_dict())
            best_val_true = deepcopy(val_true)
            best_val_preds = deepcopy(val_preds)

        print(
            f"Epoch {epoch + 1}/{num_epochs} | "
            f"Train Acc: {train_acc:.4f} | "
            f"Val Acc: {val_acc:.4f}"
        )

    all_train_accuracies.append(fold_train_accs)
```

```
    all_val_accuracies.append(fold_val_accs)

    if best_state is not None:
        model.load_state_dict(best_state)

    precision, recall, f1, _ = precision_recall_fscore_support(
        best_val_true,
        best_val_preds,
        average="binary",
        zero_division=0
    )

    cv_results.append(
        {
            "fold": fold,
            "val_accuracy": best_val_acc,
            "precision": precision,
            "recall": recall,
            "f1": f1
        }
    )

    del model

    if torch.cuda.is_available():
        torch.cuda.empty_cache()

# ============================================================
# 12. CV Summary
# ============================================================

cv_df = pd.DataFrame(cv_results)

print("\n" + "=" * 60)
print("K-FOLD CV RESULTS mean ± std")
print("=" * 60)

print(
    f"Accuracy : {cv_df['val_accuracy'].mean():.4f} ± "
    f"{cv_df['val_accuracy'].std():.4f}"
)

print(
    f"Precision: {cv_df['precision'].mean():.4f} ± "
    f"{cv_df['precision'].std():.4f}"
)
```

```
print(
    f"Recall   : {cv_df['recall'].mean():.4f} ± "
    f"{cv_df['recall'].std():.4f}"
)

print(
    f"F1-Score : {cv_df['f1'].mean():.4f} ± "
    f"{cv_df['f1'].std():.4f}"
)

print("=" * 60)

cv_csv_path = os.path.join(
    OUTPUT_DIR,
    "roberta_binary_cv_results.csv"
)

cv_df.to_csv(cv_csv_path, index=False)

print(f"  CV results saved to: {cv_csv_path}")

# ============================================================
# 13. CV Learning Curves
# ============================================================

plt.figure(figsize=(12, 6))

epochs_per_fold = num_epochs
total_epochs = k_folds * epochs_per_fold
colors = plt.cm.tab10.colors

for fold in range(k_folds):
    global_epochs = [
        e + 1 + fold * epochs_per_fold
        for e in range(epochs_per_fold)
    ]

    plt.plot(
        global_epochs,
        all_train_accuracies[fold],
        color=colors[fold],
        linestyle="-",
        marker="o",
        label="Train" if fold == 0 else "",
        alpha=0.8
    )
```

```
    plt.plot(
        global_epochs,
        all_val_accuracies[fold],
        color=colors[fold],
        linestyle="--",
        marker="x",
        label="Val" if fold == 0 else "",
        alpha=0.8
    )

plt.title("RoBERTa: Training & Validation Accuracy Across 5-Fold CV")
plt.xlabel("Epoch Global")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.6)
plt.xticks(range(1, total_epochs + 1))
plt.tight_layout()

cv_curve_path = os.path.join(
    OUTPUT_DIR,
    "roberta_binary_cv_learning_curves.png"
)

plt.savefig(cv_curve_path, dpi=300, bbox_inches="tight")
plt.show()

print(f"  CV learning curve saved to: {cv_curve_path}")

# ============================================================
# 14. Final Model Training with Internal Validation
# ============================================================

print("\nRetraining final RoBERTa model on FULL TRAIN...")

ros_full = RandomOverSampler(random_state=SEED)

X_train_res, y_train_res = ros_full.fit_resample(
    np.array(X_train_raw).reshape(-1, 1),
    y_train_raw
)

X_train_res = X_train_res.flatten().tolist()
y_train_res = y_train_res.tolist()

X_tr_final, X_val_final, y_tr_final, y_val_final = train_test_split(
    X_train_res,
    y_train_res,
```

```
    test_size=0.1,
    random_state=SEED,
    stratify=y_train_res
)

train_dataset = TextClassificationDataset(
    X_tr_final,
    y_tr_final,
    tokenizer,
    max_length
)

val_dataset = TextClassificationDataset(
    X_val_final,
    y_val_final,
    tokenizer,
    max_length
)

test_dataset = TextClassificationDataset(
    X_test,
    y_test,
    tokenizer,
    max_length
)

train_loader = DataLoader(
    train_dataset,
    batch_size=batch_size,
    shuffle=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=batch_size,
    shuffle=False
)

test_loader = DataLoader(
    test_dataset,
    batch_size=batch_size,
    shuffle=False
)

final_model = RoBERTaClassifier(
    model_name,
    num_classes
```

```
).to(device)

optimizer = AdamW(
    final_model.parameters(),
    lr=learning_rate
)

total_steps = len(train_loader) * num_epochs

scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=0,
    num_training_steps=total_steps
)

criterion = nn.CrossEntropyLoss()

final_train_losses = []
final_train_accs = []
final_val_losses = []
final_val_accs = []

best_final_val_acc = 0.0
best_final_state = None

for epoch in range(num_epochs):
    final_model.train()

    total_loss = 0.0
    correct = 0
    total = 0

    for batch in tqdm(
        train_loader,
        desc=f"Epoch {epoch + 1}/{num_epochs}"
    ):
        optimizer.zero_grad()

        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels_batch = batch["label"].to(device)

        outputs = final_model(
            input_ids,
            attention_mask
        )
```

```
        loss = criterion(
            outputs,
            labels_batch
        )

        loss.backward()

        torch.nn.utils.clip_grad_norm_(
            final_model.parameters(),
            1.0
        )

        optimizer.step()
        scheduler.step()

        total_loss += loss.item()

        preds = outputs.argmax(dim=1)

        correct += (preds == labels_batch).sum().item()
        total += labels_batch.size(0)

    avg_train_loss = total_loss / max(1, len(train_loader))
    train_acc = correct / max(1, total)

    final_train_losses.append(avg_train_loss)
    final_train_accs.append(train_acc)

    final_model.eval()

    val_loss = 0.0
    val_correct = 0
    val_total = 0

    with torch.no_grad():
        for batch in val_loader:
            input_ids = batch["input_ids"].to(device)
            attention_mask = batch["attention_mask"].to(device)
            labels_batch = batch["label"].to(device)

            outputs = final_model(
                input_ids,
                attention_mask
            )

            loss = criterion(
                outputs,
```

```
                labels_batch
            )

            val_loss += loss.item()

            preds = outputs.argmax(dim=1)

            val_correct += (preds == labels_batch).sum().item()
            val_total += labels_batch.size(0)

    avg_val_loss = val_loss / max(1, len(val_loader))
    val_acc = val_correct / max(1, val_total)

    final_val_losses.append(avg_val_loss)
    final_val_accs.append(val_acc)

    if val_acc > best_final_val_acc:
        best_final_val_acc = val_acc
        best_final_state = deepcopy(final_model.state_dict())

    print(
        f"Epoch {epoch + 1}/{num_epochs} | "
        f"Train Loss: {avg_train_loss:.4f} | "
        f"Train Acc: {train_acc:.4f} | "
        f"Val Loss: {avg_val_loss:.4f} | "
        f"Val Acc: {val_acc:.4f}"
    )

if best_final_state is not None:
    final_model.load_state_dict(best_final_state)

# ============================================================
# 15. Save Final RoBERTa Model
# ============================================================

final_model.roberta.save_pretrained(SAVE_DIR)
tokenizer.save_pretrained(SAVE_DIR)

torch.save(
    final_model.state_dict(),
    os.path.join(SAVE_DIR, "roberta_binary_classifier_state_dict.pt")
)

print(f"\n Final RoBERTa model saved to: {SAVE_DIR}")

# ============================================================
# 16. Final Model Learning Curves
```

```
# ============================================================

epochs_range = range(1, num_epochs + 1)

plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)

plt.plot(
    epochs_range,
    final_train_accs,
    "b-o",
    label="Train Acc"
)

plt.plot(
    epochs_range,
    final_val_accs,
    "r-o",
    label="Val Acc"
)

plt.title("Final RoBERTa Model: Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)

plt.plot(
    epochs_range,
    final_train_losses,
    "b-o",
    label="Train Loss"
)

plt.plot(
    epochs_range,
    final_val_losses,
    "r-o",
    label="Val Loss"
)

plt.title("Final RoBERTa Model: Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
```

```
plt.legend()
plt.grid(True)

plt.tight_layout()

final_curve_path = os.path.join(
    OUTPUT_DIR,
    "roberta_binary_final_learning_curves.png"
)

plt.savefig(final_curve_path, dpi=300, bbox_inches="tight")
plt.show()

print(f"  Final learning curves saved to: {final_curve_path}")

# ============================================================
# 17. Final Test Evaluation
# ============================================================

print("\nEvaluating on HELD-OUT TEST SET...")

final_model.eval()

test_preds = []
test_labels = []
test_probs = []

with torch.no_grad():
    for batch in tqdm(
        test_loader,
        desc="Test Evaluation"
    ):
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels_batch = batch["label"].to(device)

        outputs = final_model(
            input_ids,
            attention_mask
        )

        probs = torch.softmax(outputs, dim=1)[:, 1]
        preds = outputs.argmax(dim=1)

        test_probs.extend(probs.cpu().numpy())
        test_preds.extend(preds.cpu().numpy())
        test_labels.extend(labels_batch.cpu().numpy())
```

```
test_acc = accuracy_score(
    test_labels,
    test_preds
)

precision, recall, f1, _ = precision_recall_fscore_support(
    test_labels,
    test_preds,
    average="binary",
    zero_division=0
)

print("\nFINAL TEST RESULTS:")
print(f"Accuracy : {test_acc:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1-Score : {f1:.4f}")

print("\nClassification Report Test Set:")

print(
    classification_report(
        test_labels,
        test_preds,
        labels=[0, 1],
        target_names=["No issue", "Have_issue"],
        digits=4,
        zero_division=0
    )
)

# ============================================================
# 18. Save Test Results
# ============================================================

test_results_df = pd.DataFrame(
    {
        "true_label": test_labels,
        "pred_label": test_preds,
        "prob_have_issue": test_probs
    }
)

test_results_path = os.path.join(
    OUTPUT_DIR,
    "roberta_binary_test_predictions.csv"
```

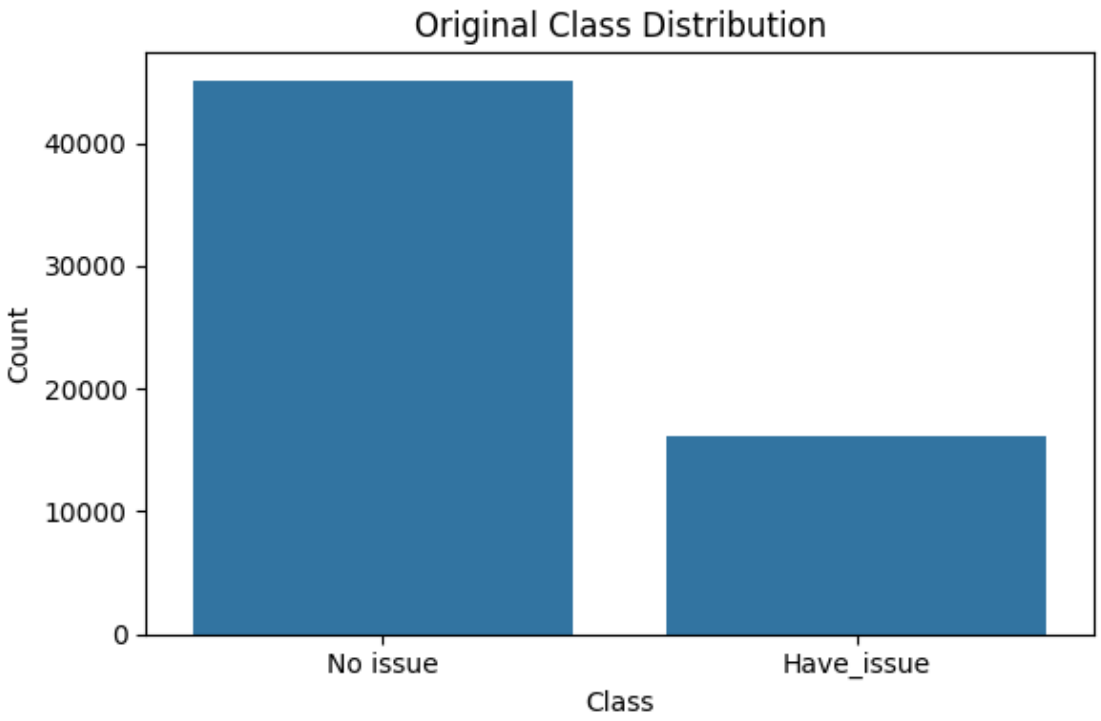
```
)

test_results_df.to_csv(
    test_results_path,
    index=False
)

print(f"  Test predictions saved to: {test_results_path}")

report_dict = classification_report(
    test_labels,
    test_preds,
    labels=[0, 1],
    target_names=["No issue", "Have_issue"],
    digits=4,
    zero_division=0,
    output_dict=True
)

report_df = pd.DataFrame(report_dict).transpose()

report_path = os.path.join(
    OUTPUT_DIR,
    "roberta_binary_classification_report.csv"
)

report_df.to_csv(
    report_path,
    index=True
)

print(f"  Classification report saved to: {report_path}")

# ============================================================
# 19. Confusion Matrix
# ============================================================

cm = confusion_matrix(
    test_labels,
    test_preds,
    labels=[0, 1]
)

cm_df = pd.DataFrame(
    cm,
    index=["No issue", "Have_issue"],
    columns=["No issue", "Have_issue"]
```

```
)

cm_csv_path = os.path.join(
    OUTPUT_DIR,
    "roberta_binary_confusion_matrix.csv"
)

cm_df.to_csv(
    cm_csv_path,
    index=True
)

print(f" Confusion matrix saved to: {cm_csv_path}")

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No issue", "Have_issue"],
    yticklabels=["No issue", "Have_issue"]
)

plt.title("RoBERTa: Confusion Matrix Final Test Set")
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.tight_layout()

cm_fig_path = os.path.join(
    OUTPUT_DIR,
    "roberta_binary_final_cm.png"
)

plt.savefig(
    cm_fig_path,
    dpi=300,
    bbox_inches="tight"
)

plt.show()

print(f" Confusion matrix figure saved to: {cm_fig_path}")

# ============================================================
# 20. ROC Curve
```

```
# ============================================================

fpr, tpr, _ = roc_curve(
    test_labels,
    test_probs
)

roc_auc = auc(
    fpr,
    tpr
)

plt.figure(figsize=(6, 5))

plt.plot(
    fpr,
    tpr,
    color="darkorange",
    lw=2,
    label=f"ROC AUC = {roc_auc:.3f}"
)

plt.plot(
    [0, 1],
    [0, 1],
    color="navy",
    lw=2,
    linestyle="--"
)

plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("RoBERTa: ROC Curve Final Test Set")
plt.legend(loc="lower right")
plt.grid(True)
plt.tight_layout()

roc_fig_path = os.path.join(
    OUTPUT_DIR,
    "roberta_binary_final_roc.png"
)

plt.savefig(
    roc_fig_path,
```

```
    dpi=300,
    bbox_inches="tight"
)

plt.show()

print(f"  ROC curve saved to: {roc_fig_path}")

# ============================================================
# 21. Final Completion Message
# ============================================================

print("\n  Full RoBERTa binary classification pipeline completed successfully.")
print(f"  Final Test Accuracy : {test_acc:.4f}")
print(f"  Final Test Precision: {precision:.4f}")
print(f"  Final Test Recall   : {recall:.4f}")
print(f"  Final Test F1-Score : {f1:.4f}")
print(f"  Final ROC AUC       : {roc_auc:.4f}")
print(f"  All outputs saved in: {OUTPUT_DIR}")
```

```
libgomp: Invalid value for environment variable OMP_NUM_THREADS
/root/.ssh/Issue_Detection_Paper_data/mytransforLearing/lib/python3.12/site-
packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm

libgomp: Invalid value for environment variable OMP_NUM_THREADS

 Found pytorch_model.bin. The code will use pytorch_model.bin.
 Using local RoBERTa model from:
/root/.ssh/Issue_Detection_Paper_data/bert_best_final_model/roberta
 Using dataset from: /root/.ssh/Issue_Detection_Paper_data/annotated_dataset_fo
r_binary_classifications.csv
Using device: cuda
 Dataset loaded. Original columns: ['Rating_Star', 'Headings', 'Base_Reviews',
'Have_issue']

Class distribution original:
Have_issue
0    45096
1    16198
Name: count, dtype: int64
```




```
  Class distribution figure saved to: /root/.ssh/Issue_Detection_Paper_data/robe
rta_binary_outputs/roberta_binary_original_class_distribution.png

Dataset Splits:
→ Full Train for CV: 52099 samples
→ Held-Out Test: 9195 samples
→ Train class raw: Counter({np.int64(0): 38331, np.int64(1): 13768})
→ Test class: Counter({0: 6765, 1: 2430})
  Local RoBERTa tokenizer loaded successfully

Hyperparameters:
Model path: /root/.ssh/Issue_Detection_Paper_data/bert_best_final_model/roberta
Number of classes: 2
Max length: 128
Batch size: 16
Epochs: 6
Learning rate: 1e-05

Starting 5-Fold CV…

==================================================
FOLD 1/5
==================================================
Fold 1 → Train: 61328 oversampled, Val: 10420
```

```
→ Train classes: Counter({0: 30664, 1: 30664})
→ Val classes: Counter({np.int64(0): 7667, np.int64(1): 2753})

Loading weights: 100%|     | 199/199 [00:00<00:00, 68269.79it/s]
[transformers] RobertaModel LOAD REPORT from:
/root/.ssh/Issue_Detection_Paper_data/bert_best_final_model/roberta
Key                        | Status     |  |
---------------------------+------------+--+-
lm_head.dense.bias         | UNEXPECTED |  |
lm_head.layer_norm.weight  | UNEXPECTED |  |
lm_head.decoder.weight     | UNEXPECTED |  |
lm_head.bias               | UNEXPECTED |  |
lm_head.dense.weight       | UNEXPECTED |  |
lm_head.layer_norm.bias    | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture;
not ok if you expect identical arch.

Epoch 1/6 | Train Acc: 0.8168 | Val Acc: 0.8915
Epoch 2/6 | Train Acc: 0.8982 | Val Acc: 0.9016
Epoch 3/6 | Train Acc: 0.9241 | Val Acc: 0.9260
Epoch 4/6 | Train Acc: 0.9455 | Val Acc: 0.9275
Epoch 5/6 | Train Acc: 0.9588 | Val Acc: 0.9340
Epoch 6/6 | Train Acc: 0.9664 | Val Acc: 0.9346

==================================================
FOLD 2/5
==================================================
Fold 2 → Train: 61330 oversampled, Val: 10420
→ Train classes: Counter({0: 30665, 1: 30665})
→ Val classes: Counter({np.int64(0): 7666, np.int64(1): 2754})

Loading weights: 100%|     | 199/199 [00:00<00:00, 39276.58it/s]
[transformers] RobertaModel LOAD REPORT from:
/root/.ssh/Issue_Detection_Paper_data/bert_best_final_model/roberta
Key                        | Status     |  |
---------------------------+------------+--+-
lm_head.dense.bias         | UNEXPECTED |  |
lm_head.layer_norm.weight  | UNEXPECTED |  |
lm_head.decoder.weight     | UNEXPECTED |  |
lm_head.bias               | UNEXPECTED |  |
lm_head.dense.weight       | UNEXPECTED |  |
lm_head.layer_norm.bias    | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture;
not ok if you expect identical arch.

Epoch 1/6 | Train Acc: 0.8213 | Val Acc: 0.8808
```

```
Epoch 2/6 | Train Acc: 0.8997 | Val Acc: 0.9061
Epoch 3/6 | Train Acc: 0.9261 | Val Acc: 0.9100
Epoch 4/6 | Train Acc: 0.9443 | Val Acc: 0.9179
Epoch 5/6 | Train Acc: 0.9556 | Val Acc: 0.9309
Epoch 6/6 | Train Acc: 0.9647 | Val Acc: 0.9358

==================================================
FOLD 3/5
==================================================
Fold 3 → Train: 61330 oversampled, Val: 10420
→ Train classes: Counter({1: 30665, 0: 30665})
→ Val classes: Counter({np.int64(0): 7666, np.int64(1): 2754})

Loading weights: 100%|      | 199/199 [00:00<00:00, 60933.46it/s]
[transformers] RobertaModel LOAD REPORT from:
/root/.ssh/Issue_Detection_Paper_data/bert_best_final_model/roberta
Key                         | Status     |  |
----------------------------+------------+--+-
lm_head.dense.bias          | UNEXPECTED |  |
lm_head.layer_norm.weight | UNEXPECTED |  |
lm_head.decoder.weight      | UNEXPECTED |  |
lm_head.bias                | UNEXPECTED |  |
lm_head.dense.weight        | UNEXPECTED |  |
lm_head.layer_norm.bias     | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture;
not ok if you expect identical arch.

Epoch 1/6 | Train Acc: 0.8212 | Val Acc: 0.8697
Epoch 2/6 | Train Acc: 0.8977 | Val Acc: 0.8991
Epoch 3/6 | Train Acc: 0.9260 | Val Acc: 0.9239
Epoch 4/6 | Train Acc: 0.9448 | Val Acc: 0.9200
Epoch 5/6 | Train Acc: 0.9572 | Val Acc: 0.9262
Epoch 6/6 | Train Acc: 0.9657 | Val Acc: 0.9301

==================================================
FOLD 4/5
==================================================
Fold 4 → Train: 61330 oversampled, Val: 10420
→ Train classes: Counter({0: 30665, 1: 30665})
→ Val classes: Counter({np.int64(0): 7666, np.int64(1): 2754})

Loading weights: 100%|      | 199/199 [00:00<00:00, 58816.61it/s]
[transformers] RobertaModel LOAD REPORT from:
/root/.ssh/Issue_Detection_Paper_data/bert_best_final_model/roberta
Key                         | Status     |  |
----------------------------+------------+--+-
lm_head.dense.bias          | UNEXPECTED |  |
```

```
lm_head.layer_norm.weight | UNEXPECTED |  |
lm_head.decoder.weight    | UNEXPECTED |  |
lm_head.bias              | UNEXPECTED |  |
lm_head.dense.weight      | UNEXPECTED |  |
lm_head.layer_norm.bias   | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture;
not ok if you expect identical arch.

Epoch 1/6 | Train Acc: 0.8247 | Val Acc: 0.8687
Epoch 2/6 | Train Acc: 0.9009 | Val Acc: 0.8921
Epoch 3/6 | Train Acc: 0.9275 | Val Acc: 0.9245
Epoch 4/6 | Train Acc: 0.9446 | Val Acc: 0.9352
Epoch 5/6 | Train Acc: 0.9567 | Val Acc: 0.9327
Epoch 6/6 | Train Acc: 0.9664 | Val Acc: 0.9378

==================================================
FOLD 5/5
==================================================
Fold 5 → Train: 61330 oversampled, Val: 10419
→ Train classes: Counter({0: 30665, 1: 30665})
→ Val classes: Counter({np.int64(0): 7666, np.int64(1): 2753})

Loading weights: 100%|     | 199/199 [00:00<00:00, 58978.70it/s]
[transformers] RobertaModel LOAD REPORT from:
/root/.ssh/Issue_Detection_Paper_data/bert_best_final_model/roberta
Key                       | Status     |  |
--------------------------+------------+--+-
lm_head.dense.bias        | UNEXPECTED |  |
lm_head.layer_norm.weight | UNEXPECTED |  |
lm_head.decoder.weight    | UNEXPECTED |  |
lm_head.bias              | UNEXPECTED |  |
lm_head.dense.weight      | UNEXPECTED |  |
lm_head.layer_norm.bias   | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture;
not ok if you expect identical arch.

Epoch 1/6 | Train Acc: 0.8246 | Val Acc: 0.8564
Epoch 2/6 | Train Acc: 0.9005 | Val Acc: 0.9097
Epoch 3/6 | Train Acc: 0.9265 | Val Acc: 0.9269
Epoch 4/6 | Train Acc: 0.9447 | Val Acc: 0.9247
Epoch 5/6 | Train Acc: 0.9590 | Val Acc: 0.9299
Epoch 6/6 | Train Acc: 0.9669 | Val Acc: 0.9379

============================================================
K-FOLD CV RESULTS mean ± std
```

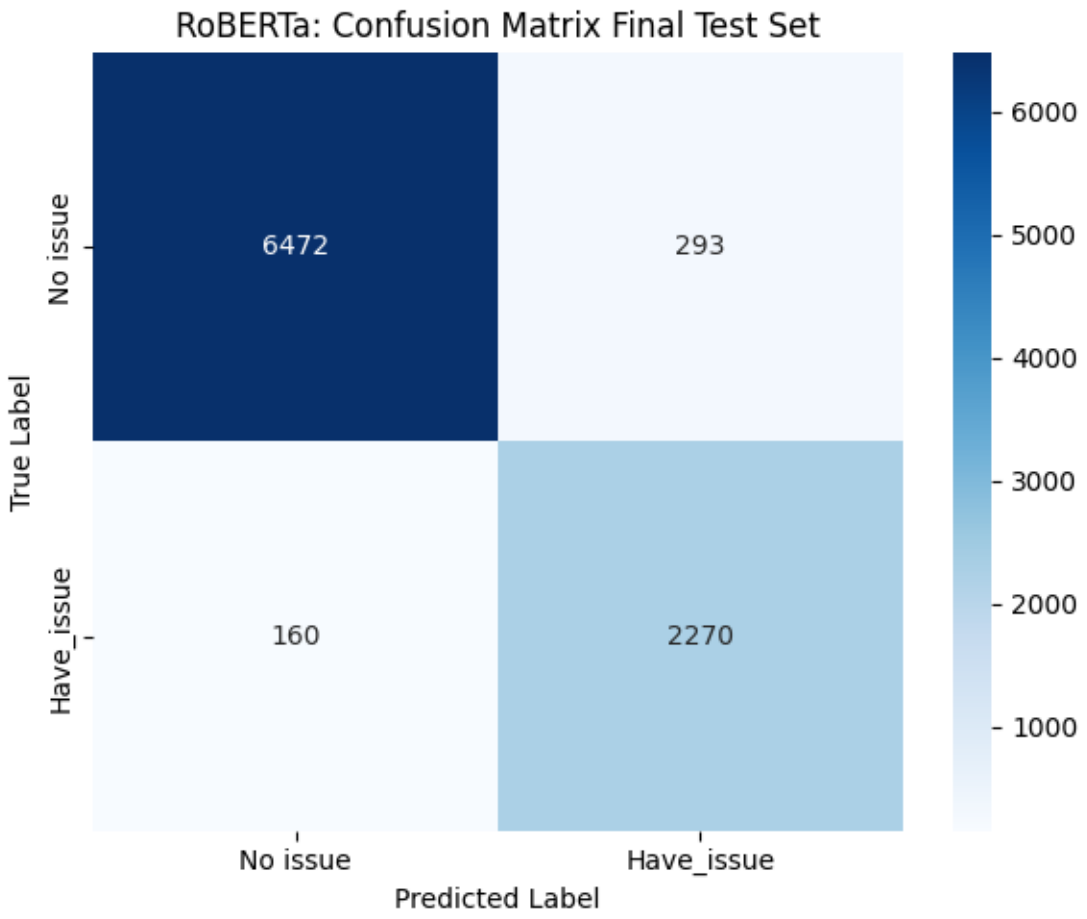
```
============================================================
Accuracy : 0.9353 ± 0.0032
Precision: 0.8365 ± 0.0120
Recall   : 0.9387 ± 0.0076
F1-Score : 0.8846 ± 0.0045
============================================================
  CV results saved to: /root/.ssh/Issue_Detection_Paper_data/roberta_binary_outp
uts/roberta_binary_cv_results.csv
```

```
  CV learning curve saved to: /root/.ssh/Issue_Detection_Paper_data/roberta_bina
ry_outputs/roberta_binary_cv_learning_curves.png

Retraining final RoBERTa model on FULL TRAIN…
Loading weights: 100%|      | 199/199 [00:00<00:00, 65939.84it/s]
[transformers] RobertaModel LOAD REPORT from:
/root/.ssh/Issue_Detection_Paper_data/bert_best_final_model/roberta
Key                         | Status     |  |
----------------------------+------------+--+-
lm_head.dense.bias          | UNEXPECTED |  |
lm_head.layer_norm.weight   | UNEXPECTED |  |
lm_head.decoder.weight      | UNEXPECTED |  |
lm_head.bias                | UNEXPECTED |  |
lm_head.dense.weight        | UNEXPECTED |  |
lm_head.layer_norm.bias     | UNEXPECTED |  |

Notes:
- UNEXPECTED:    can be ignored when loading from different task/architecture;
not ok if you expect identical arch.
Epoch 1/6: 100%|      | 4313/4313 [04:58<00:00, 14.43it/s]
```

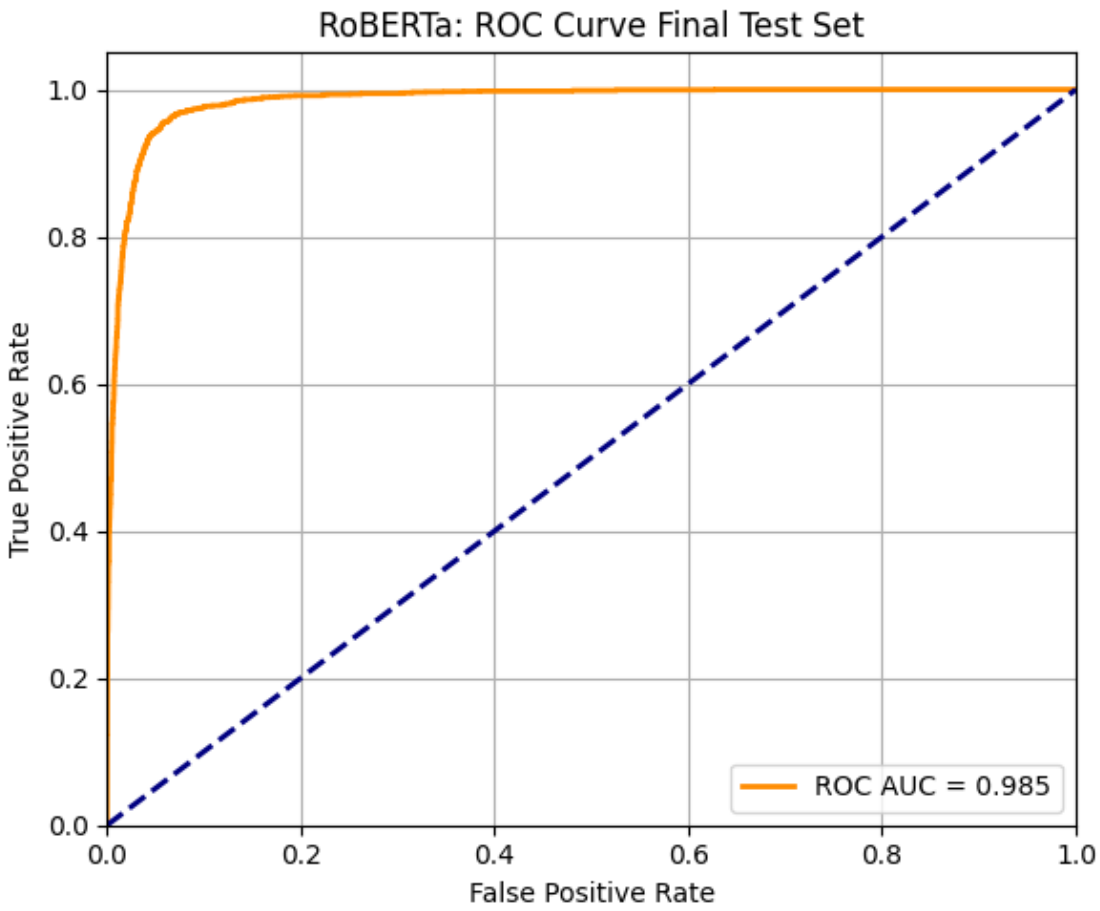
```
Epoch 1/6 | Train Loss: 0.3985 | Train Acc: 0.8213 | Val Loss: 0.2673 | Val Acc:
0.9000

Epoch 2/6: 100%|      | 4313/4313 [04:56<00:00, 14.56it/s]

Epoch 2/6 | Train Loss: 0.2533 | Train Acc: 0.9006 | Val Loss: 0.2375 | Val Acc:
0.9144

Epoch 3/6: 100%|      | 4313/4313 [04:56<00:00, 14.53it/s]

Epoch 3/6 | Train Loss: 0.2030 | Train Acc: 0.9281 | Val Loss: 0.1890 | Val Acc:
0.9412

Epoch 4/6: 100%|      | 4313/4313 [04:48<00:00, 14.95it/s]

Epoch 4/6 | Train Loss: 0.1735 | Train Acc: 0.9437 | Val Loss: 0.1943 | Val Acc:
0.9517

Epoch 5/6: 100%|      | 4313/4313 [03:58<00:00, 18.05it/s]

Epoch 5/6 | Train Loss: 0.1514 | Train Acc: 0.9560 | Val Loss: 0.1816 | Val Acc:
0.9598

Epoch 6/6: 100%|      | 4313/4313 [04:13<00:00, 17.02it/s]

Epoch 6/6 | Train Loss: 0.1322 | Train Acc: 0.9641 | Val Loss: 0.1907 | Val Acc:
0.9594

Writing model shards: 100%|      | 1/1 [00:00<00:00,  2.16it/s]


  Final RoBERTa model saved to:
/root/.ssh/Issue_Detection_Paper_data/roberta_binary_final_model
```

```
  Final learning curves saved to: /root/.ssh/Issue_Detection_Paper_data/roberta_
binary_outputs/roberta_binary_final_learning_curves.png

Evaluating on HELD-OUT TEST SET…

Test Evaluation: 100%|      | 575/575 [00:07<00:00, 79.45it/s]
```

```
FINAL TEST RESULTS:
Accuracy : 0.9507
Precision: 0.8857
Recall   : 0.9342
F1-Score : 0.9093

Classification Report Test Set:
              precision    recall  f1-score   support

    No issue     0.9759    0.9567    0.9662      6765
  Have_issue     0.8857    0.9342    0.9093      2430

    accuracy                         0.9507      9195
   macro avg     0.9308    0.9454    0.9377      9195
weighted avg     0.9520    0.9507    0.9511      9195

  Test predictions saved to: /root/.ssh/Issue_Detection_Paper_data/roberta_binar
y_outputs/roberta_binary_test_predictions.csv
  Classification report saved to: /root/.ssh/Issue_Detection_Paper_data/roberta_
binary_outputs/roberta_binary_classification_report.csv
  Confusion matrix saved to: /root/.ssh/Issue_Detection_Paper_data/roberta_binar
y_outputs/roberta_binary_confusion_matrix.csv
```



Confusion matrix figure saved to: /root/.ssh/Issue_Detection_Paper_data/roberta_binary_outputs/roberta_binary_final_cm.png




```
  ROC curve saved to: /root/.ssh/Issue_Detection_Paper_data/roberta_binary_outpu
ts/roberta_binary_final_roc.png

  Full RoBERTa binary classification pipeline completed successfully.
  Final Test Accuracy : 0.9507
  Final Test Precision: 0.8857
  Final Test Recall   : 0.9342
  Final Test F1-Score : 0.9093
  Final ROC AUC       : 0.9848
  All outputs saved in:
/root/.ssh/Issue_Detection_Paper_data/roberta_binary_outputs
```